GUIA VISUAL 052

VARIÁVEIS E O AMBIENTE

*O script de backup roda em quatro segundos no terminal. No `cron`, às duas da manhã, falha todas as noites com `nmap: command not found`. Nada no script mudou — mudou **o ambiente**.*

052
GUIAS LINUX

A SINTAXE IMPLACÁVEL · E O QUE O FILHO HERDA

✓ `nome="kali"`
✗ `nome = "kali"` vira comando
✗ `nome= "kali"` roda `kali`
✓ `${nome}_2024` chaves = fronteira
✗ `$nome_2024` outra variável, vazia
✓ `MAIÚSCULA` ambiente/constante
✓ `minúscula` local do script

SHELL PAI · PID 4021

`TOKEN="abc"`
variável do shell — só aqui

`export CAMINHO="/opt"`
variável de ambiente

`printenv` só as exportadas · `set` tudo · `declare -p` com atributos

→ fork + exec
cópia do ambiente
✗ nunca de volta

FILHO · PID 4088

`$CAMINHO` → `/opt` herdada
`$TOKEN` → **vazia** não exportada
`CAMINHO="/tmp"` altera só a cópia — o pai não vê
Prefixo temporário não suja a sessão: `LC_ALL=C date`

QUAIS ARQUIVOS CADA SHELL LÊ — E POR QUE O `CRON` NÃO LÊ NENHUM

SHELL DE LOGIN
console, ssh, su -
`/etc/profile`
`/etc/profile.d/*.sh`
`~/.bash_profile`
`~/.profile` (que chama o `.bashrc`)

INTERATIVO, SEM LOGIN
nova aba do terminal
`/etc/bash.bashrc`
`~/.bashrc`
e mais nada — o resto vem **herdado**

NÃO INTERATIVO
script, cron, systemd
nenhum arquivo lido
PATH mínimo (/usr/bin:/bin)
sem TERM, sem aliases

A coluna vermelha é a resposta ao mistério: o `.bashrc` **nunca foi lido**, logo o `PATH` que você acrescentou lá não existe. Correções, em ordem: **1.** caminho absoluto para cada comando · **2.** `export PATH=...` explícito no topo do script · **3.** `PATH=` no próprio *crontab*. Chamar o `.bashrc` de dentro do script é a saída tentadora e **errada** — ele tem código de sessão interativa e pode abortar cedo. Para a máquina inteira: `/etc/environment` (pares simples, lido pelo PAM) ou um fragmento em `/etc/profile.d/`.

VAZIO × NÃO DEFINIDO · AS ESPECIAIS QUE O BASH PREENCHE

EXPANSÃO	COMPORTAMENTO
`${VAR:-padrao}`	**Não definida OU vazia** → usa o padrão. É a que se usa 90% das vezes.
`${VAR-padrao}`	Sem os dois-pontos: só se **não definida**. Vazia continua vazia.
`${VAR:=padrao}`	Usa **e atribui** o padrão à variável.
`: "${VAR:?msg}"`	**Aborta com a sua mensagem** se faltar. O `:` é o comando nulo, só força a avaliação.
`${VAR:-}`	Sob `set -u`, a forma de dizer "sei que pode não existir".

`$?`	saída do último comando
`$$`	PID do shell · `$PPID` do pai
`$0`	nome do script
`$#`	quantos argumentos
`$@`	cada argumento uma palavra
`$*`	todos como uma só
`$!`	PID do último em 2º plano
`$LINENO`	linha atual — para mensagem de erro
`$SECONDS`	`$((SECONDS-inicio))` cronometra
`$RANDOM`	0–32767, **não** criptográfico

```
kali@kali: ~/lab-vars
$ LOCAL="so-aqui"; export GLOBAL="vai-junto"
$ bash -c 'echo "no filho: LOCAL=[$LOCAL] GLOBAL=[$GLOBAL]"'
no filho: LOCAL=[] GLOBAL=[vai-junto]   # sem export, o filho nao ve
$ env -i /bin/bash ./backup.sh    # simula o cron: ambiente vazio, falha na hora
$ echo "$SHELL"; ps -p $$ -o comm= # o de LOGIN x o REALMENTE em execucao
/usr/bin/zsh
bash
$ declare -i n=0; n="3*7"; echo "$n"   # -i: atribuicao vira conta -> 21
$ readonly CONF=/etc/app.conf          # erro de digitacao vira erro na hora
```

Segredo em variável de ambiente é segredo exposto. O ambiente de um processo vive em `/proc/PID/environ`, legível pelo dono e pelo root — e é **herdado por todo filho**, inclusive o instalador de pacote que resolveu mandar telemetria. Por argumento (`--senha s3nh4`) é pior ainda: aparece no `ps aux` para qualquer usuário da máquina. Em ordem: **1.** arquivo `600` lido com `source` · **2.** entrada padrão (`--password-stdin`) · **3.** gerenciador (`pass`, `secret-tool`) · **4.** prefixo de um comando só, nunca `export` na sessão. E **nunca** segredo dentro de script versionado: apagar a linha num commit posterior não apaga nada do histórico.

RESUMO RÁPIDO

- ✓ **Sem espaço em volta do `=`.** Com espaço, `nome` vira nome de comando — daí o `command not found` que engana quem vem de outra linguagem.
- ✓ **`export` é a fronteira** entre variável do shell e de ambiente. A herança é por **cópia** e de **mão única**: o filho altera só o retrato que recebeu.
- ✓ **Shell não interativo não lê arquivo nenhum.** `PATH` explícito no topo ou caminho absoluto — a correção do `cron`.
- ✓ **`SHELL`** é o shell de **login** do `/etc/passwd`, não o em execução (`ps -p $$ -o comm=`). E `LANG` muda a saída dos comandos: script que interpreta saída fixa `LC_ALL=C`.
- ✓ **Vazio ≠ não definido.** `${VAR:-x}` cobre os dois; `${VAR:?msg}` aborta cedo com mensagem sua.

NO KALI

O `sudo` limpa o ambiente — e isso quebra script. O `env_reset` está ligado no `/etc/sudoers`: um `export CHAVE=v` seguido de `sudo meu.sh` chega ao script **sem** a chave. Saídas: `sudo CHAVE=v comando`, `sudo -E` (preserva *tudo* — cuidado) ou `env_keep` via `visudo`. **O Kali põe `/usr/sbin` e `/sbin` no `PATH` de qualquer usuário**, porque quase toda ferramenta de rede vive ali — por isso `tcpdump` e `iptables` completam com Tab sem `sudo`, e depois falham com `Operation not permitted`; compare `printenv PATH` antes e depois de `sudo -i`. Conheça as variáveis das ferramentas: `PROXYCHAINS_CONF_FILE`, `HTTP_PROXY` / `NO_PROXY`, `GOPATH` — proxy configurado no `~/.profile` e esquecido é a origem clássica de "a rede parou depois do reboot". Em laboratório, `HISTCONTROL=ignorespace` mantém credencial de teste fora do `~/.bash_history` (no Zsh, `setopt HIST_IGNORE_SPACE`). E em **live USB sem persistência** tudo que você exportou some no desligamento: use `PATH` explícito e caminho absoluto, a mesma disciplina do `cron`.